RÉPUBLIQUE ALGÉRIENNE DÉMOCRATIQUE ET POPULAIRE
MINISTÈRE DE L'ENSEIGNEMENT SUPÉRIEUR ET DE LA RECHERCHE SCIENTIFIQUE
ÉCOLE NATIONALE SUPÉRIEURE D'INFORMATIQUE

Rapport de TP VCL N°2

2ème Année Cycle Supérieur (2CS)
Option : Systèmes Informatiques (SIQ)

Virtualisation sous Linux avec KVM

Réalisé par :

ATTIA Oussama Abderraouf
SRAICH Imene

Encadré par :

MENNACER Djamel-Eddine

Année universitaire : 2025–2026

Contents

Introduction **3**

1 Utilisation de QEMU / KVM sous Linux **4**

1.1 Préparation de l'environnement sous VMware Workstation 4

1.1.1 Préparation des machines virtuelles 4

1.1.2 Installation et configuration des paramètres IP 5

1.2 Création du réseau virtuel pour KVM 5

1.3 Installation de KVM 6

1.3.1 Installation et configuration 6

1.3.2 Vérification de la configuration 6

1.4 Création et gestion des machines virtuelles sur le Client KVM 7

1.4.1 Création de VM1 (Ubuntu) avec virt-manager 7

1.4.2 Création de VM2 (RedHat) avec virt-install 8

1.4.3 Gestion des machines virtuelles avec virsh 8

2 Utilisation de l'API libvirt avec Python **9**

Conclusion **14**

Introduction

KVM (Kernel-based Virtual Machine) est un module de virtualisation intégré directement dans le noyau Linux. Considéré comme un hyperviseur de type 1, il utilise l'émulateur QEMU pour émuler les architectures des processeurs des systèmes invités et exécuter les instructions sur le système hôte. Pour gérer facilement cette virtualisation, QEMU/KVM utilise l'API **libvirt**, qui fonctionne comme un daemon sous Linux et offre divers outils de gestion.

Ce TP est divisé en deux parties :

- **Partie A :** Création et gestion de machines virtuelles sur l'hyperviseur KVM en utilisant divers outils (virt-manager, virt-install, virt-viewer, etc.).
- **Partie B :** Conception d'un script Python basé sur la bibliothèque `libvirt` pour gérer programmatiquement ces machines virtuelles.

Chapter 1 : Utilisation de QEMU / KVM sous Linux

1.1 Préparation de l'environnement sous VMware Workstation

Pour atteindre les objectifs du TP, deux machines virtuelles principales sont créées et configurées au sein de VMware Workstation.

1.1.1 Préparation des machines virtuelles

1. Serveur KVM

Cette machine a pour système d'exploitation CentOS 7.0 et intègre le module KVM, jouant ainsi le rôle d'un hyperviseur de type 1.

Périphérique	**Configuration**
Mémoire (RAM)	4 GB
Processeurs	2
Disque Dur (SCSI)	40 GB (Préalloué)
CD/DVD (IDE)	Utilisation de l'ISO CentOS 7 64-bit
Adaptateur Réseau 1	Custom (VMnet10)
Adaptateur Réseau 2	Custom (VMnet10)
Système d'exploitation	Linux (CentOS 7 64-bit)

Table 1.1: Configuration de la machine Serveur KVM

2. Client KVM

Cette machine fonctionne également sous CentOS 7.0. Elle joue le rôle de client de gestion pour administrer les machines virtuelles installées sur le Serveur KVM.

Périphérique	Configuration
Mémoire (RAM)	1 GB
Processeurs	2
Disque Dur (SCSI)	10 GB (Préalloué)
CD/DVD (IDE)	Utilisation de l'ISO CentOS 7 64-bit
Adaptateur Réseau	Custom (VMnet10)
Système d'exploitation	Linux (CentOS 7 64-bit)

Table 1.2: Configuration de la machine Client KVM

Remarques importantes

- Les adaptateurs réseaux des deux machines sont configurés en mode **Host-only** et connectés à **VMnet10** (réseau virtuel créé précédemment).
- Les processeurs des machines *Serveur KVM* et *Client KVM* doivent supporter la virtualisation imbriquée. Il est impératif de cocher la case : **Virtualize Intel VT-x/EPT or AMD-V/RVI** dans les paramètres du CPU sur VMware.

1.1.2 Installation et configuration des paramètres IP

Une fois les systèmes d'exploitation installés, nous devons configurer leurs adresses IP de manière statique via l'interface graphique (Wired > IPv4 > Manual) :

- **Machine KVM_Server :**
  - Adresse IP : `10.10.0.1`
  - Masque : `255.255.255.0`
  - Passerelle : `10.10.0.1`
  - DNS : `10.10.0.1`
- **Machine KVM_Client :**
  - Adresse IP : `10.10.0.100`
  - Masque : `255.255.255.0`
  - Passerelle : `10.10.0.1`
  - DNS : `10.10.0.1`

1.2 Création du réseau virtuel pour KVM

Sur la machine **Serveur KVM**, nous allons créer un pont réseau (bridge) en exécutant les commandes suivantes :

```
# Lister les connexions reseau actuelles
[root@kvm1 ~]# nmcli c

# Ajouter un bridge reseau "kvmbr0" sur l'interface physique "ens33"
[root@kvm1 ~]# nmcli c add type bridge autoconnect yes con-name kvmbr0 ifname kvmbr0
[root@kvm1 ~]# nmcli c delete ens33
[root@kvm1 ~]# nmcli c add type bridge-slave autoconnect yes con-name ens33 ifname ens33 master kvmbr0
[root@kvm1 ~]# systemctl restart NetworkManager

# Verifier la creation du reseau virtuel
[root@kvm1 ~]# nmcli c
```

Lister les connexions et créer le bridge

Le bridge `kvmbr0` est maintenant créé et lié à l'interface physique `ens33`.

1.3 Installation de KVM

1.3.1 Installation et configuration

Sur les **deux** machines (Serveur KVM et Client KVM), exécutez les commandes suivantes dans l'ordre (une connexion internet est requise temporairement) :

```
# Mettre a jour le systeme et installer les paquets
[root@kvm1 ~]# yum -y update
[root@kvm1 ~]# yum -y install qemu-kvm libvirt virt-install virt-viewer virt-manager

# Activer le module KVM
[root@kvm1 ~]# modprobe kvm

# Demarrer et activer le daemon de libvirt
[root@kvm1 ~]# systemctl start libvirtd
[root@kvm1 ~]# systemctl enable libvirtd
```

Installation des paquets KVM

1.3.2 Vérification de la configuration

Pour s'assurer que l'installation s'est bien déroulée, exécutez ces vérifications sur le Serveur KVM :

```
# Verifier que la virtualisation materielle est activee (VT-x)
[root@kvm1 ~]# lscpu | grep Virtualization

```

```
# Verifier le chargement correct du module KVM
[root@kvm1 ~]# lsmod | grep kvm

# Verifier que le service libvirt est en cours d'execution
[root@kvm1 ~]# systemctl status libvirtd
```

Verification de l'environnement KVM

1.4 Création et gestion des machines virtuelles sur le Client KVM

L'objectif est de créer et de gérer les machines virtuelles hébergées sur le **Serveur KVM** à distance, depuis la machine **Client KVM**, en se connectant via SSH.

Sur la machine **Client KVM**, installez d'abord ce paquet :

```
[root@client1 ~]# yum -y install openssh-askpass
```

1.4.1 Création de VM1 (Ubuntu) avec virt-manager

1. Ouvrez l'outil graphique en tapant `virt-manager` dans le terminal du Client KVM.
2. Cliquez sur **File** > **Add Connection**.
3. Saisissez les paramètres SSH pour vous connecter au Serveur KVM :
   - **Hypervisor :** QEMU/KVM
   - **Method :** SSH
   - **Username :** root
   - **Hostname :** `10.10.0.1`
4. Entrez le mot de passe root du Serveur KVM lorsqu'il est demandé.
5. Une fois connecté, faites un clic droit sur la connexion distante (`qemu+ssh://root@10.10.0.1`) et cliquez sur **New Virtual Machine**.
6. **Installation locale :** Choisissez l'image ISO Ubuntu (ex: `ubuntu-20.04-desktop.iso`).
7. **Ressources :** Attribuez 2048 MB de RAM et 2 CPUs.
8. **Disque :** Créez un disque local de 10 GB.
9. **Réseau :** Nommez la machine **VM1** et dans la sélection du réseau, choisissez le bridge que nous avons créé : **Bridge kvmbr0**.
10. Cliquez sur Finish. La machine s'ouvrira dans `virt-viewer`. Procédez à l'installation d'Ubuntu.
11. Une fois Ubuntu installé, attribuez-lui l'adresse IP `10.10.0.101`.

1.4.2 Création de VM2 (RedHat) avec virt-install

Nous allons créer la deuxième VM en ligne de commande depuis le Client KVM :

```
[root@client1 ~]# virt-install \
  --connect qemu+ssh://root@10.10.0.1/system \
  --virt-type kvm \
  --name VM2 \
  --os-type linux \
  --os-variant rhel7.2 \
  --cdrom /ISO/rhel-server-7.2-x86_64-DVD.iso \
  --disk
     path=/var/lib/libvirt/images/VM2.qcow2,format=qcow2,size=10
     \
  --memory 1024 \
  --vcpus 2 \
  --network bridge=kvmbr0
```

Creation de la VM2 en CLI

Introduisez le mot de passe root du Serveur KVM. Une fenêtre `virt-viewer` s'ouvrira pour finaliser l'installation de RedHat. Une fois terminée, attribuez l'adresse IP `10.10.0.102` à la VM2.

1.4.3 Gestion des machines virtuelles avec virsh

`virsh` est un outil en ligne de commande très puissant pour interagir avec libvirt. Depuis le **Client KVM**, connectez-vous au serveur via :

```
[root@client1 ~]# virsh --connect
   qemu+ssh://root@10.10.0.1/system
```

Une fois dans l'invite interactive `virsh #`, vous pouvez exécuter :

```
virsh # list --all          # Lister toutes les VMs (allumees et
   eteintes)
virsh # start VM1           # Demarrer la VM1
virsh # shutdown VM1        # Arreter proprement la VM1
virsh # dominfo VM1         # Afficher des infos generales
   (etat, RAM, CPU)
virsh # domstats VM1        # Afficher des statistiques avancees
   de la VM
virsh # domiflist VM1       # Afficher les interfaces reseaux de
   la VM
virsh # version             # Afficher la version de libvirt/QEMU
virsh # nodeinfo            # Afficher les informations CPU/RAM
   du serveur hote
```

Commandes utiles virsh

Chapter 2 : Utilisation de l'API libvirt avec Python

Dans cette partie, nous créons un script Python interactif pour interagir directement avec l'API `libvirt`.

Remarques :

- Ce script est conçu pour être exécuté **directement sur le Serveur KVM** (connexion locale).
- Il est écrit en **Python 2.7.5** et utilise les modules `libvirt` et `os`.

Voici le code source du script d'administration développé :

```python
#!/usr/bin/env python2
# -*- coding: utf-8 -*-

import libvirt
import os

''' ***************************************************
Declaration des fonctions et des structures de donnees
*************************************************** '''

# Fonction qui liste les choix du menu principal
def listeChoix():
    print("  0) Informations sur la machine hyperviseur")
    print("  1) Lister les machines virtuelles")
    print("  2) Informations sur une machine virtuelle")
    print("  3) Operations sur une machine virtuelle")
    print("  4) Visualiser une machine virtuelle")
    print("  5) Quitter")

# Dictionnaire qui contient les operations possibles
operations = {
    1 : "demarrer",
    2 : "arreter",
    3 : "forcer l'arret",
    4 : "suspendre",
    5 : "reprendre"
}

# Dictionnaire qui contient les etats possibles d'une VM
state = {
    libvirt.VIR_DOMAIN_RUNNING : "en cours d'execution",
```

```
    libvirt.VIR_DOMAIN_PAUSED : "mise en pause",
    libvirt.VIR_DOMAIN_SHUTDOWN : "en arret",
    libvirt.VIR_DOMAIN_SHUTOFF : "eteinte"
}

# Fonction qui donne l'etat d'une machine virtuelle
def etat(n):
    try:
        return state.get(n)
    except:
        return "eteinte"

''' ************************************************************
Ne pas afficher les messages d'erreur de libvirt sur la console
************************************************************ '''
def libvirt_callback(userdata, err):
    pass

libvirt.registerErrorHandler(f=libvirt_callback, ctx=None)

''' ******************************
Ouvrir une connexion a l'hyperviseur
****************************** '''
# Connexion locale en mode systeme
conn = libvirt.open("qemu:///system")
if not conn:
    raise SystemExit("La connexion a l'hyperviseur a echoue !")

''' ******************
Boucle du menu principal
****************** '''
print("Programme de gestion des machines virtuelles ; veuillez
   entrer votre choix")
listeChoix()

choix = -1

while (choix != 5):
    try:
        choix = int(raw_input("\nVotre choix : "))
    except:
        choix = -1

    if(choix == 0):
        print("\n  Informations sur la machine hyperviseur :")
        print("  ----------------------------------------")
        print("  Nom de la machine : " +
           str(conn.getHostname()))
        print("  Max processus virtuels : " +
           str(conn.getMaxVcpus(None)))
        print("  Modele du processeur : " +
```

```
            str(conn.getInfo()[0]))
        print("  Taille de la RAM : " + str(conn.getInfo()[1]) + " Mo")
        print("  Processeurs actifs : " + str(conn.getInfo()[2]))
        print("  Frequence du processeur : " + str(conn.getInfo()[3]) + " MHz")
        print("  Noeuds NUMA : " + str(conn.getInfo()[4]))
        print("  Cœurs par processeurs : " + str(conn.getInfo()[6]))

    elif(choix == 1):
        list_VM = conn.listAllDomains()
        if len(list_VM) == 0:
            max_len = 0
        else:
            max_len = len(max(list_VM, key = lambda d : len(d.name())).name())

        print("\n  Liste des machines virtuelles :\n")
        espace = min(max(max_len, 20), 20)
        nom = "Nom".ljust(espace)
        head = "Id    " + nom + "  Etat"
        print("  " + head)
        print("  " + "".rjust(len(head), '-'))

        for dom in list_VM:
            id = str(dom.ID())
            if dom.ID() == -1:
                id = '-'
            print("  " + id.ljust(6) + dom.name().ljust(espace+4) + etat(dom.state()[0]))

    elif(choix == 2):
        nom = raw_input("  Introduire le nom d'une machine : ")
        try:
            dom = conn.lookupByName(nom)
        except libvirt.libvirtError as e:
            print("  Erreur : Aucun domaine n'a ce nom !")
        else:
            print("\n  Informations sur '" + nom + "' :")
            print("  ID : " + str(dom.ID()))
            print("  Etat : " + etat(dom.info()[0]))
            print("  Max RAM : " + str(dom.info()[1]/1024) + " Mo")
            print("  RAM utilisee : " + str(dom.info()[2]/1024) + " Mo")
            print("  CPUs virtuels : " + str(dom.info()[3]))

            if not dom.isActive():
```

```
                print("  Adresse IP : indisponible (machine eteinte)")
            else:
                try:
                    iffaces = dom.interfaceAddresses(libvirt.VIR_DOMAIN_INTERFACE_A
                    0)
                except libvirt.libvirtError:
                    print("  Adresse IP : QEMU Guest Agent non installe/connecte.")
                else:
                    adresse = ""
                    for (name, val) in iffaces.iteritems():
                        if val['addrs'] and name != 'lo':
                            for ipaddr in val['addrs']:
                                if ipaddr['type'] == libvirt.VIR_IP_ADDR_TYPE_IPV4:
                                    adresse += ipaddr['addr'] + " "
                    print("  Adresse IP : " + str(adresse))

    elif(choix == 3):
        print("  Choisir l'operation a effectuer :")
        print("    1) Demarrer\n    2) Arreter\n    3) Forcer l'arret\n    4) Suspendre\n    5) Reprendre")
        try:
            op = int(raw_input("  Operation : "))
        except:
            op = -1

        if(op in {1, 2, 3, 4, 5}):
            nom = raw_input("  Introduire le nom de la machine : ")
            try:
                dom = conn.lookupByName(nom)
            except:
                print("  Erreur : Aucun domaine trouve.")
            else:
                try:
                    if(op == 1 and dom.state()[0] == libvirt.VIR_DOMAIN_SHUTOFF):
                        dom.create()
                        print("  Machine demarree avec succes.")
                    elif(op == 2 and dom.state()[0] == libvirt.VIR_DOMAIN_RUNNING):
                        dom.shutdown()
                        print("  Machine en cours d'arret.")
                    elif(op == 3 and dom.state()[0] != libvirt.VIR_DOMAIN_SHUTOFF):
                        dom.destroy()
```

```
                        print("  Machine arretee de force.")
                    elif(op == 4 and dom.state()[0] !=
                       libvirt.VIR_DOMAIN_SHUTOFF):
                        dom.suspend()
                        print("  Machine suspendue.")
                    elif(op == 5 and dom.state()[0] ==
                       libvirt.VIR_DOMAIN_PAUSED):
                        dom.resume()
                        print("  Machine reprise.")
                except libvirt.libvirtError as e:
                    print("  Erreur : " + str(e))

    elif(choix == 4):
        nom = raw_input("  Nom de la machine a visualiser : ")
        try:
            dom = conn.lookupByName(nom)
        except:
            print("  Erreur : Machine inexistante.")
        else:
            try:
                os.system("virt-viewer " + nom + " &")
            except:
                print("  Erreur de lancement virt-viewer.")

    elif(choix != 5):
        print("  Choix errone !")

    if (choix in [0, 1, 2, 3, 4]):
        raw_input("\nAppuyez sur 'Enter' pour continuer...")
        listeChoix()

conn.close()
```

Script de gestion des machines virtuelles (Python 2)

Conclusion

Ce TP nous a permis d'apprendre à mettre en place un environnement de virtualisation performant en utilisant l'hyperviseur intégré au noyau Linux, KVM/QEMU. Nous avons exploré diverses manières de gérer les machines virtuelles :

- **Mode graphique** : via `virt-manager` et `virt-viewer`.
- **Ligne de commandes** : via `virt-install` et `virsh`.
- **Programmation logicielle** : en développant un script sur mesure en Python s'appuyant sur l'API `libvirt`.

Ces acquis consolident nos compétences pratiques en administration de systèmes virtualisés sous Linux et ouvrent la voie à l'automatisation avancée des infrastructures Cloud.